

[S6] Pattern Mining I

[attachment:58e868dc-45ee-4055-8ecf-1db4d6ddae12:20250425111504_6._Session6_-_Pattern_Mining_I.pdf](#)

1. Konsep Dasar Frequent Pattern

Frequent pattern adalah pola (seperti set item, sub-sekuens, atau sub-struktur) yang muncul dengan frekuensi tinggi dalam suatu kumpulan data. Konsep ini pertama kali diusulkan oleh Agrawal, Imielinski, dan Swami pada tahun 1993 untuk analisis data keranjang belanja (*basket data analysis*).

Mengapa Frequent Pattern Penting?

- **Inherent Regularities:** Membantu menemukan keteraturan alami dalam data.
- **Fondasi Data Mining:** Menjadi landasan bagi tugas data mining lainnya seperti analisis asosiasi, korelasi, kausalitas, hingga klasifikasi dan analisis klaster.
- **Aplikasi Luas:** Digunakan dalam desain katalog, analisis kampanye penjualan, analisis log web, hingga analisis sekuens DNA.

2. Parameter Utama: Support dan Confidence

Dalam penambangan aturan asosiasi (Association Rules), terdapat dua metrik utama yang digunakan untuk mengukur kualitas pola:

- **Support (\$s\$):** Probabilitas bahwa sebuah transaksi mengandung gabungan itemset \$X\$ dan \$Y\$.
 - **Absolute Support:** Frekuensi atau jumlah kemunculan itemset.
 - **Relative Support:** Fraksi/persentase transaksi yang mengandung itemset tersebut.

- **Confidence (\$c\$):** Probabilitas kondisional bahwa transaksi yang memiliki itemset \$X\$ juga akan mengandung itemset \$Y\$.

Contoh: Jika aturan $\{Beer\} \rightarrow \{Diaper\}$ memiliki support 60% dan confidence 100%, artinya dalam 60% dari seluruh transaksi, Beer dan Diaper dibeli bersamaan, dan 100% orang yang membeli Beer pasti membeli Diaper.

3. Optimasi Pola: Closed dan Max-Patterns

Menghasilkan semua pola frekuen bisa menyebabkan ledakan jumlah data (kombinatorial). Untuk mengatasinya, digunakan dua teknik kompresi:

- **Closed Pattern:** Sebuah itemset yang frekuen dan tidak ada super-pattern (pola yang lebih besar) di atasnya yang memiliki nilai support yang sama. Ini adalah bentuk kompresi tanpa kehilangan informasi (*lossless*).
- **Max-pattern:** Sebuah itemset yang frekuen dan tidak ada super-pattern di atasnya yang juga bersifat frekuen.

4. Algoritma Apriori

Apriori adalah algoritma klasik yang menggunakan pendekatan **candidate generation and test**.

Prinsip Utama (Downward Closure)

Algoritma ini bekerja berdasarkan prinsip: **"Jika sebuah itemset bersifat tidak frekuen, maka semua super-set (turunannya) juga tidak akan frekuen"**.

Sebaliknya, setiap subset dari itemset yang frekuen haruslah frekuen.

Langkah-langkah Kerja:

1. **Scan Database:** Mencari itemset ukuran 1 (L_1) yang memenuhi *minimum support*.
2. **Self-Joining:** Menggabungkan L_k dengan dirinya sendiri untuk menghasilkan kandidat C_{k+1} .
3. **Pruning (Pemangkasan):** Menghapus kandidat yang memiliki subset yang tidak frekuen.

4. **Repeat:** Proses berulang sampai tidak ada lagi kandidat atau pola frekuen yang ditemukan.
-

5. Pengembangan dan Perbaikan Apriori

Karena algoritma Apriori membutuhkan pemindaian database berkali-kali dan menghasilkan banyak kandidat, muncul beberapa metode peningkatan:

- **Partitioning:** Membagi database menjadi beberapa partisi. Pola yang frekuen secara global haruslah frekuen di setidaknya satu partisi. Metode ini hanya membutuhkan dua kali pemindaian database.
 - **DHP (Direct Hashing and Pruning):** Menggunakan tabel *hash* untuk mengurangi jumlah kandidat itemset ukuran 2.
 - **Sampling:** Mengambil sampel database untuk menemukan pola, lalu memverifikasi hasilnya pada database utama.
 - **DIC (Dynamic Itemset Counting):** Menambahkan itemset kandidat baru "di tengah jalan" saat pemindaian database masih berlangsung untuk mengurangi jumlah pemindaian.
-

Kesimpulan dari Latihan (Exercise)

- **Contoh Pola Berguna:** Menemukan bahwa pelanggan yang membeli sereal sering membeli susu (cross-marketing).
 - **Pentingnya Minimum Support:**
 - Jika **terlalu tinggi**, kita mungkin melewatkan pola menarik yang jarang muncul namun signifikan.
 - Jika **terlalu rendah**, jumlah pola yang dihasilkan akan meledak secara eksponensial, membebani komputasi, dan menghasilkan banyak "noise".
-

1. Kenapa keberadaan frequent pattern dalam data transaksi dapat dianggap sebagai representasi perilaku kolektif manusia bukan sekedar kemunculan

statistik item secara berulang?

2. Jika seluruh frequent item set dapat diperoleh menggunakan apriori, kenapa pendekatan closed pattern mining masih dianggap memiliki nilai representasi pengetahuan yang lebih efisien tanpa kehilangan informasi penting?

Jawaban:

1. kenapa? → "karena"

pola bukan hanya sebab-akibat, atau antara kausalitas dan kausasiti
representasi pengetahuan itu sifatnya adalah proyeksi, klo soal begini
harusnya jawabnya pake
"karena data ini di ciptakan/di emisi oleh manusia"

2. kenapa? → "karena"

closed pattern ketika ...

misal abcd

dan bcd

dan keduanya punya support yang sama, pilih abcd aja

"karena superset yang ada di closed pattern bisa digunakan untuk
merekonstruksi subset yang ada di ..."

BASIC CONCEPTS

Frequent pattern

- A pattern (a set of items, subsequences, substructures, etc.) that occurs frequently in a dataset
- An intrinsic and important property of datasets
- First proposed by Agrawal, Imielinski, and Swami [AIS93] in the context of [frequent itemsets](#) and [association rule mining](#)
- Motivation: finding inherent regularities in data
- Applications: basket data analysis, cross-marketing, catalog design, sale campaign analysis, Web log (click stream) analysis, and DNA sequence analysis.

pola bisa di katakan sebuah pola bila:

- berulang
- bisa diletakan dalam sebuah himpunan
- punya properties internal

inherent contohnya:

- oh dia sama sama segitiga
- oh dia sama sama menginginkan kedamaian

motivasi: mencari pola umum dari kesamaan properties internal yang ada pada data

BASIC CONCEPTS (2)

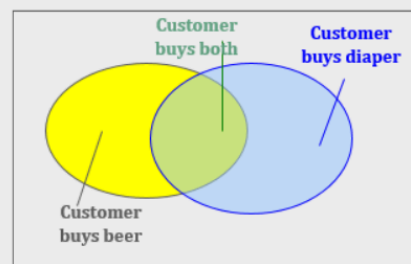
- Foundation for many essential data mining tasks
 - Association, correlation, and causality analysis
 - Sequential, structural (e.g., sub-graph) patterns
 - Pattern analysis in spatiotemporal, multimedia, time-series, and stream data
 - Classification: discriminative, frequent pattern analysis
 - Cluster analysis: frequent pattern-based clustering
 - Data warehousing: iceberg cube and cube-gradient
 - Semantic data compression: fascicles
 - Broad applications An intrinsic and important property of datasets

stream data ada distribution shift

BASIC CONCEPTS (3)

- Example:
 - **itemset**: a set of one or more items
 - **k-itemset** $X = \{x_1, \dots, x_k\}$
 - **(absolute) support**, or, **support count** of X : frequency or occurrence of an itemset X
 - **(relative) support**, s , is the fraction of transactions that contains X (i.e., the probability that a transaction contains X)
 - An itemset X is **frequent** if X 's support is no less than a minsup threshold

Tid	Items bought
10	Beer, Nuts, Diaper
20	Beer, Coffee, Diaper
30	Beer, Diaper, Eggs
40	Nuts, Eggs, Milk
50	Nuts, Coffee, Diaper, Eggs, Milk



BASIC CONCEPTS (4)

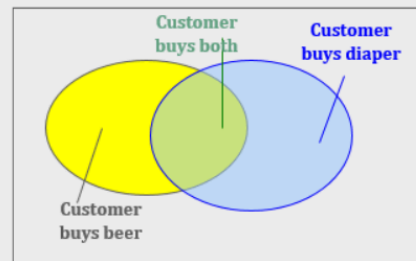
- Find all the rules $X \rightarrow Y$ with minimum support and confidence
 - support, s, probability** that a transaction contains $X \cup Y$
 - confidence, c, conditional probability** that a transaction having X also contains Y

Let minsup = 50%, minconf = 50%

Freq. Pat.: Beer:3, Nuts:3, Diaper:4, Eggs:3, {Beer, Diaper}:3

- Association rules: (many more!)
 - Beer $\rightarrow$ Diaper (60%, 100%)
 - Diaper $\rightarrow$ Beer (60%, 75%)

Tid	Items bought
10	Beer, Nuts, Diaper
20	Beer, Coffee, Diaper
30	Beer, Diaper, Eggs
40	Nuts, Eggs, Milk
50	Nuts, Coffee, Diaper, Eggs, Milk



itemset: semua item yang unig

k-itemset: subset itemset

support or support count: frequency data

klo berbicara frequency itu hubungannya dengan distribusi

klo testset evaluationnya ga selalu ... berdistribusi

takutnya testsetnya di asumsikan distribusi, makanya pake support, biar ga terjadi perbedaan makna

- Support (s):** Probabilitas bahwa sebuah transaksi mengandung gabungan itemset X dan Y .
- Absolute Support:** Jumlah atau frekuensi kemunculan suatu itemset dalam database.
- Relative Support:** Fraksi atau persentase transaksi yang mengandung itemset tersebut dibanding total seluruh transaksi.
- Confidence (c):** Probabilitas kondisional yang menunjukkan seberapa sering item Y muncul dalam transaksi yang sudah mengandung item X .

BASIC CONCEPTS (5)

Closed patterns and max-patterns

- A long pattern contains a combinatorial number of sub-patterns, e.g., $\{a_1, \dots, a_{100}\}$ contains $(100^1) + (100^2) + \dots + (100^{100}) = 2^{100} - 1 = 1.27 * 10^{30}$ sub-patterns!
- Solution: Mine *closed patterns* and *max-patterns* instead
- An itemset X is **closed** if X is *frequent* and there exists *no super-pattern* $Y \supset X$, with the same support as X (proposed by Pasquier, et al. @ ICDT'99)
- An itemset X is a **max-pattern** if X is frequent and there exists no frequent super-pattern $Y \supset X$ (proposed by Bayardo @ SIGMOD'98)
- Closed pattern is a lossless compression of freq. patterns
 - Reducing the # of patterns and rules

dari belajar ini, dan bisa dapet confidence tinggi → enak untuk pengetahuan bisnis, tapi...

klo banyak bisa jadi kutukan time nya meledak

jika X itu frequent (ketika sering muncul, kapan? ketika $\geq$ minimum support) dan tidak ada super-pattern Y dari X (tapi ga ada Y yang punya support kayak X)

BASIC CONCEPTS (6)

- Example:

$DB = \{ \langle a_1, \dots, a_{100} \rangle, \langle a_1, \dots, a_{50} \rangle \},$

$Min_sup = 1$

Solutions:

- Closed itemset:

$\langle a_1, \dots, a_{100} \rangle = 1$

$\langle a_1, \dots, a_{50} \rangle = 2$

- Max-pattern

$\langle a_1, \dots, a_{100} \rangle = 1$

- All patterns: !!

secara memory bagus max pattern

secara informasi bagus closed item

BASIC CONCEPTS (7)

Computational complexity of frequent itemset mining

- How many itemsets are potentially to be generated in the worst case?
 - The number of frequent itemsets to be generated is sensitive to the minsup threshold
 - When minsup is low, there exist potentially an exponential number of frequent itemsets
 - The worst case: M^N where M: # distinct items, and N: max length of transactions
- The worst case complexity vs. the expected probability
 - Ex. Suppose Walmart has 10^4 kinds of products
 - The chance to pick up one product 10^{-4}
 - The chance to pick up a particular set of 10 products: $\sim 10^{-40}$
 - What is the chance this particular set of 10 products to be frequent 10^3 times in 10^9 transactions?

FREQUENT ITEMSET MINING METHODS

Downward closure properties

- Any subset of a frequent itemset must be frequent
- If **{beer, diaper, nuts}** is frequent, so is **{beer, diaper}**
- i.e., every transaction having {beer, diaper, nuts} also contains {beer, diaper}

Scalable mining methods

- Apriori (Agrawal & Srikant@VLDB'94)
- Freq. pattern growth (FPgrowth—Han, Pei & Yin @SIGMOD'00)
- Vertical data format approach (Charm—Zaki & Hsiao @SDM'02)

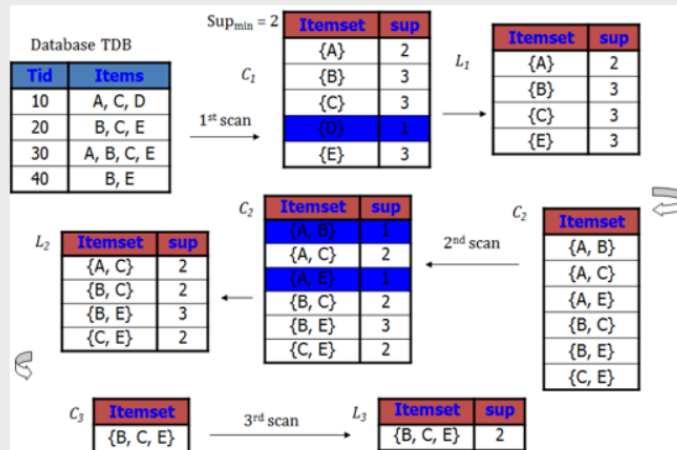
APRIORI ALGORITHM

A candidate generation and test approach

- Apriori pruning principle: if there is any itemset which is infrequent, its superset should not be generated/tested! (Agrawal & Srikant @VLDB'94, Mannila, et al. @KDD'94)
- Method:
 - Initially, scan DB once to get frequent 1-itemset
 - Generate length (k+1) candidate itemsets from length k frequent itemsets
 - Test the candidates against DB
 - Terminate when no frequent or candidate set can be generated

APRIORI ALGORITHM (2)

- Example:



1. scan semua item, cari berapa support semua item
2. bandingkan dengan minimal support, misal min-supp = 2
3. yang selain 2 di exclude dari candidates
4. perbesar: berdasarkan database dengan menimbang list sebelumnya

...

...

...



APRIORI ALGORITHM (3)

Pseudocode

C_k : Candidate itemset of size k

L_k : frequent itemset of size k

$L_1 = \{\text{frequent items}\};$

for ($k = 1; L_k \neq \emptyset; k++$) **do begin**

C_{k+1} = candidates generated from L_k ;

for each transaction t in database **do**

 increment the count of all candidates in C_{k+1} that are contained in t

L_{k+1} = candidates in C_{k+1} with min_support

end

return $\cup_k L_k$;



APRIORI ALGORITHM (4)

Implementation

- Generate candidates
 - Step 1: self-joining L_k
 - Step 2: pruning
- Example of candidate-generation
 - $L_3 = \{abc, abd, acd, ace, bcd\}$
 - Self-joining: $L_3 * L_3$
 - abcd from abc and abd
 - acde from acd and ace
- Pruning: acde is removed because ade is not in L_3
 - $C_4 = \{abcd\}$

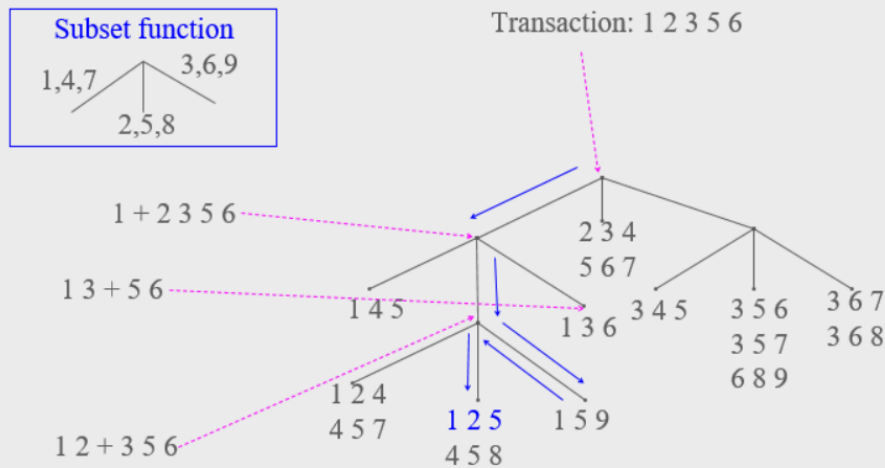
APRIORI ALGORITHM (5)

Count supports of candidates

- Why counting supports of candidates a problem?
 - The total number of candidates can be very huge
 - One transaction may contain many candidates
- Method:
 - Candidate itemsets are stored in a hash-tree
 - Leaf node of hash-tree contains a list of itemsets and counts
 - Interior node contains a hash table
 - Subset function: finds all the candidates contained in a transaction

APRIORI ALGORITHM (6)

Count supports of candidates using Hash Tree



APRIORI ALGORITHM (7)

Candidate generation: an SQL implementation

- SQL implementation of candidate generation
 - Suppose the items in L_{k-1} are listed in an order
 - Step 1: self-joining L_{k-1}

```

insert into  $C_k$ 
select  $p.item_1, p.item_2, ..., p.item_{k-1}, q.item_{k-1}$ 
from  $L_{k-1} p, L_{k-1} q$ 
where  $p.item_1 = q.item_1, ..., p.item_{k-2} = q.item_{k-2}, p.item_{k-1} < q.item_{k-1}$ 

```
 - Step 2: pruning


```

for all itemsets  $c$  in  $C_k$  do
  for all (k-1)-subsets  $s$  of  $c$  do
    if ( $s$  is not in  $L_{k-1}$ ) then delete  $c$  from  $C_k$ 

```
- Use object-relational extensions like UDFs, BLOBs, and Table functions for efficient implementation

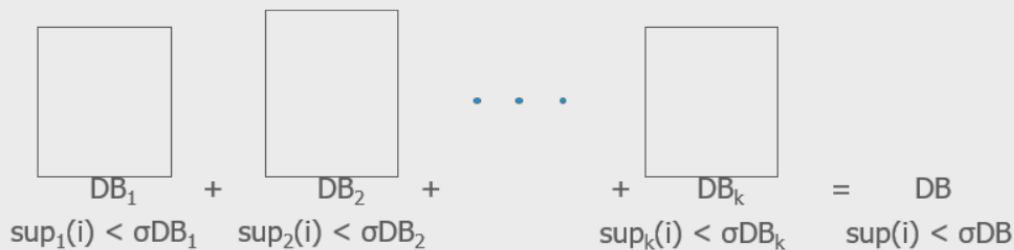
FURTHER IMPROVEMENT OF THE APRIORI METHOD

- Major computational challenges
 - Multiple scans of transaction database
 - Huge number of candidates
 - Tedious workload of support counting for candidates
- Improving Apriori: general ideas
 - Reduce passes of transaction database scans
 - Shrink number of candidates
 - Facilitate support counting of candidates

FURTHER IMPROVEMENT OF THE APRIORI METHOD (2)

Partition: scan database only twice

- Any itemset that is potentially frequent in DB must be frequent in at least one of the partitions of DB
 - Scan 1: partition database and find local frequent patterns
 - Scan 2: consolidate global frequent patterns
- A. Savasere, E. Omiecinski and S. Navathe, VLDB'95



setiap partisi punya support masing masing

FURTHER IMPROVEMENT OF THE APRIORI METHOD (3)

DHP: reduce the number of candidates

- A k-itemset whose corresponding hashing bucket count is below the threshold cannot be frequent
 - Candidates: a, b, c, d, e
 - Hash entries
 - {ab, ad, ae}
 - {bd, be, de}
 - ...
 - Frequent 1-itemset: a, b, d, e
 - ab is not a candidate 2-itemset if the sum of count of {ab, ad, ae} is below support threshold
- J. Park, M. Chen, and P. Yu. [An effective hash-based algorithm for mining association rules](#). SIGMOD'95

Count	Itemsets
35	{ab, ad, ae}
88	{bd, be, de}
.	.
.	.
.	.
102	{yz, gs, wt}

Hash Table

cara biar kandidat ga terlalu banyak?

FURTHER IMPROVEMENT OF THE APRIORI METHOD (4)

Sampling for frequent patterns

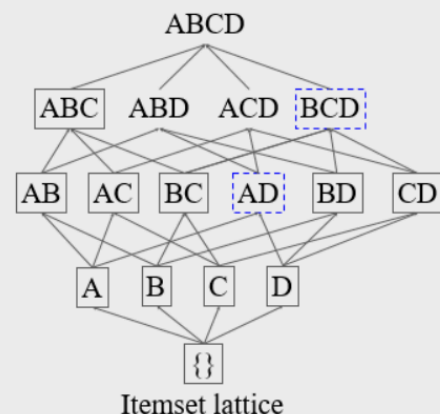
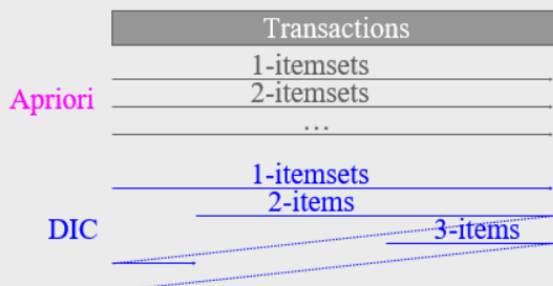
- Select a sample of original database, mine frequent patterns within sample using Apriori
- Scan database once to verify frequent itemsets found in sample, only **borders** of closure of frequent patterns are checked
 - Example: check abcd instead of ab, ac, ..., etc.
- Scan database again to find missed frequent patterns
- H. Toivonen. [Sampling large databases for association rules](#). In *VLDB'96*

selain itu kita bisa sampling data, tapi harus hati hati, takutnya kehilangan distribusinya, sampling biasanya hanya untuk stage awal

FURTHER IMPROVEMENT OF THE APRIORI METHOD (5)

DIC: Reduce Number of Scans

- Once both A and D are determined frequent, the counting of AD begins
- Once all length-2 subsets of BCD are determined frequent, the counting of BCD begins



ini hanya menghitung ... yang frequent

karena A D freq → baru di hitung berapa banyak

hanya hitung untuk item yang prior combinationnya udah di definisikan frequent

problem DIC: klo data candidate banyak → memory meledak, makanya mainkan aja storagenya di partition, jadi method" ini tuh bisa di combine



SUMMARY

- Basic concepts: association rules, support-confident framework, closed and max-patterns
- Scalable frequent pattern mining methods
 - Apriori (Candidate generation & test)

numerik bisa pake apriori tapi harus didiskritisasi